DO NOW — RECALL

Linked lists with arrays and pointers

Name: _____ Class: _____ Date: _____

1. What is the main difference between an array and a list?

2. In an array, what does an index identify?

3. In a linked list, what does null in a node's next pointer mean?

4. If the first item is at index 4 and its next pointer is 1, where is the second item?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

An array has a fixed size; a list can change size.

2. Model answer

The position used to access an array element.

3. Model answer

There is no next item.

4. Model answer

At index 1.

By the end, you can…

• represent a linked list using parallel arrays and pointer indexes

• traverse a list by following pointers rather than adjacent indexes

• add and remove items by changing pointer values

Vocabulary — know these terms

Recall the term — linked list

Recall the term — pointer

Recall the term — start pointer

Recall the term — next pointer

Recall the term — null

Recall the term — traverse

Recall the term — insert

Recall the term — remove

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking.

In an array-based linked list, the item at the next array index is not necessarily the next logical item.

Circle: Confident / Getting there / Need help

Explain what you can now do independently.

GREEN PEN — Correct and explain

Choose one error. Write the correct method and explain precisely why your first response was wrong.

NEW LEARNING

Read, annotate and explain

The pointer chooses the next index

```
string[] data = {"Ben", "Daisy", "Alex", "Cara"};
int[] next = {3, -1, 0, 1};
int start = 2;
```

Run it: Alex → Ben → Cara → Daisy. The array positions stay fixed; the pointers create the logical order.

Start pointer

The first logical item is stored at index 2, not necessarily index 0.

Follow next pointers

Use -1 as null in code. Each pointer decides the next index.

Insert at the front

The new item points to the old first item, then start changes.

Remove after current

The pointer bypasses one node. The array data itself does not need to shift.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Traverse Alex → Ben → Cara → Daisy

Use the start pointer

```
current = 2
data[2] = "Alex"
```

Read next[2]

```
next[2] = 0
current = 0
data[0] = "Ben"
```

Continue pointers

```
next[0] = 3
data[3] = "Cara"

next[3] = 1
data[1] = "Daisy"
```

Reach null

```
next[1] = -1
stop
```


YOU DO — INDEPENDENT PRACTICE

Questions · Part 2 · continued

Combine, test and explain
Combine traversal, insertion and deletion using the pointer arrays. Record expected and actual outputs for the tests below.
CODE:
Empty list: start=-1
One node: next[start]=-1
Insert into a spare slot
Delete head and middle nodes
Reject insertion if no slot is free
Use -1 only as the null marker. Reuse a removed slot only after marking it free in your chosen allocation scheme.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

Question 6

Insert a new first node
Reset to the starter. Store Zoe in unused slot 4. Link it to the old head, then update start. Traverse to verify.
CODE:
// New node -> old head; then move start.

Question 7

Remove the second node
Reset to the starter. Bypass the second logical node by changing the head node's next pointer. Do not shift any data values.
CODE:
// Check the successor exists before bypassing it.

Question 8

Remove the first node safely
Reset to the starter. If the list is nonempty, move start to next[start]. Explain what happens for a one-node list.
CODE:
if (start != -1) { /* update head */ }

Question 9

Search by following links
Reset to the starter. Search for a name using current and next pointers. Report its array index or -1; test a present and absent name.
CODE:
// Follow links, not current++.

Question 10

WE DO — GUIDED PRACTICE

Try, discuss and improve

1. Access the second linked-list item

OCR asks how the second item is reached using pointer values.

```
firstIndex = start
secondIndex = ?
secondData = data[?]
```

Method / working:

2. Traverse

Use the start pointer and follow next until -1.

```
int current = ?;
while (current != -1)
{
    Console.WriteLine(data[current]);
    current = ?;
}
```

Method / working:

3. Insert at front

newIndex is an unused array slot. Change the links, then traverse again.

```
next[newIndex] = ?;
start = ?;
```

Method / working:

4. Remove second item

Bypass the item immediately after start , then run the traversal to prove the change.

```
next[start] = ?;
```

Method / working:

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

Question 1

Identify the first logical item
Run the starter. State the start index and first item. Explain why index 0 is not necessarily the head.
CODE:

```
using System;
public class Program
{
public static void Main()
{
string[] data = {"Ben", "Daisy", "Alex", "Cara", "", ""};
int[] next = {3, -1, 0, 1, -1, -1};
int start = 2; // slots 4 and 5 are unused
Console.WriteLine(data[start]);
}
}
```

Question 2

Follow the next pointers
Use the unchanged starter. Write the sequence of indexes and names visited from start until -1.

Question 3

Write the traversal loop
Replace the final print with a loop. Initialise current from start; output data[current]; follow next[current] until -1.
CODE:

```
current = start;
while (current != -1) { /* output, then follow link */ }
```

Question 4

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1 · continued

Count reachable nodes
Add a counter to your traversal. Count reachable nodes, not data.Length, which includes unused capacity.
CODE:

```
int count = 0;
// increment once per reachable node
```

Question 5

Insert after the head
Reset to the starter. Put Elliot in spare slot 4 and insert it immediately after Alex. Preserve the successor link before changing next[start].
CODE:

```
// First preserve the old successor in next[4].
// Then reconnect the predecessor.
```